

ProgTutor: A Robotic-Based Framework to Support Teaching and Learning of Programming Fundamentals

Javier Ortega-Morla , Antonio Leis , Alma Mallo , Laura Morán-Fernández , Sara Guerreiro ,
Alejandro Paz-López , Beatriz Pérez-Sánchez , Noelia Sánchez-Maróño , Alejandro Rodríguez-Arias ,
Oscar Fontenla-Romero , and Francisco Bellas 

Abstract—The initial version of ProgTutor, a learning framework designed to teach the fundamentals of computer programming in a personalized and applied manner, is presented here. The main contribution of this tool is the integration of an adaptive learning system with a 3-D robotic simulator, used to face realistic challenges in a user-friendly fashion. ProgTutor provides automated evaluations and feedback on coding errors, ensuring that learners receive the support they need to progress effectively. In addition, it features dynamic learning paths tailored to each student's pace, offloading tasks such as automated evaluation and adaptive sequencing to the tool so that students and teachers can concentrate on judgment. ProgTutor also enhances the teachers' capacities as educators, as they can focus their attention on those students with more learning difficulties. Therefore, it functions as intelligence augmentation rather than automation, with teachers remaining in the decision loop. This article introduces the conceptual and functional design of ProgTutor, and it includes piloting results with high school students during the academic course 2023–2024, focused on their acceptability of the tool and on the analysis of the real impact that this type of system could have over the formal educational landscape in the future.

Index Terms—Adaptive learning system, educational robotics, intelligence augmentation (IA), intelligent tutoring system (ITS), judgment skills, personalized learning, programming fundamentals.

I. INTRODUCTION

THE goal of reaching effective personalized learning by means of artificial intelligence (AI) tools has a long tradition in the field of AI in education (AIEd) [1]. Different names have been used to refer to this type of software, such as adaptive learning systems [2], personalized learning environments [1], AI-based learning assistants [3], or intelligent tutoring systems (ITSs) [4]. As detailed by Kabudi et al. [2], their main difference is whether they are designed to model the human tutor or the learner, but, in practice, they all aim for more personalized and autonomous learning through AI techniques.

We can find such AI-based systems for different subjects or topics like mathematics, language, or science, which have provided successful results in pilot studies [5]. They are not fully integrated into formal education systems yet, as the AI regulations are new and long-term analysis is required [6]. However, with the widespread adoption of digital technology at educational centers, it is clear that this is a key moment for the development of this field. Consequently, new research and development are necessary to understand the consequences of these systems for transforming the current learning landscapes.

In this sense, a promising approach to improve existing AI-based personalized learning systems is to include emergent visualization technologies, such as 3-D simulations or virtual reality, with the aim of providing a more realistic and practical experience to students [7], [8], taking advantage of the advance of these emerging technologies. Consequently, a learning-by-doing approach, typically used in hands-on subjects that can be tested in real-world settings, can be integrated into a digital system, expanding the range of subjects that could benefit from such simulated environments [9], [10].

The current work is focused on this scope, and it presents the initial version of *ProgTutor*, a learning framework designed to teach the fundamentals of computer programming in a personalized and applied manner. The main contribution of ProgTutor is the integration of an adaptive learning system with a 3-D robotic simulator, used to face realistic challenges by means of simple programs. The work presented here introduces the

Received 30 October 2024; revised 16 April 2025, 21 July 2025, and 30 July 2025; accepted 4 August 2025. Date of publication 12 August 2025; date of current version 26 August 2025. This work was supported in part by Grant TED2021-131172B-I00 funded by MCIN/AEI/10.13039/501100011033 and by the “EU NextGenerationEU/PRTR” and in part by the European Union (EU) through the FEDER Galicia 2021-27 Operational Program under Grant ED431G 2023/01. The work of researchers of LIDIA group was supported in part by the Xunta de Galicia under Grant ED431C 2022/44 and in part by the EU European Regional Development Fund. The Centre for Information and Communications Technology Research, as a center accredited for excellence within the Galician University System and a member of the CIGUS Network, receives subsidies from the Department of Education, Science, Universities, and Vocational Training of the Xunta de Galicia. (Corresponding author: Francisco Bellas.)

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Ethics Committee for Research and Teaching Universidade da Coruña. The research project “Tutor de programación inteligente mediante robótica simulada (ProgTutor)” was assessed in an ordinary session on the 18th of December of 2023, considering that, according to the submitted documents, the research project adequately fulfils the relevant ethical and legal aspects for research and complies with the UDC Code of Research Ethics and UDC Guide to Good Research Practice.

The authors are with the Centre for Information and Communications Technology Research, Universidade da Coruña, 15071 A Coruña, Spain (e-mail: javier.ortegal@udc.es; antonio.leis@udc.es; alma.mallo@udc.es; laura.moranf@udc.es; sara.guerreiro@udc.es; alejandro.paz.lopez@udc.es; beatriz.perez@udc.es; noelia.sanchez@udc.es; alejandro.rodriguez.arias@udc.es; oscar.fontenla@udc.es; francisco.bellas@udc.es).

Digital Object Identifier 10.1109/TLT.2025.3598041

conceptual and functional design of ProgTutor, and it includes piloting results obtained with high school students during the academic course 2023–2024, focused on their acceptability of the tool and on the expected impact that this type of system could have on the educational landscape.

Throughout this article, we frame ProgTutor explicitly as an intelligence augmentation (IA) system [11]: the machine performs high-frequency reckoning (calculations, automated evaluation, and adaptive sequencing), while educators and learners focus on human judgment (deliberation, ethics, and situation-appropriate decisions). This human-in-the-loop design aims for overall team performance that exceeds either human or AI alone.

In this study, we address the following research questions.

- RQ1 How does student engagement evolve when using ProgTutor, and what factors (e.g., simulator interaction and adaptive difficulty) drive sustained motivation?
- RQ2 To what extent does ProgTutor facilitate teachers' ability to detect learning difficulties and offer tailored support to students?
- RQ3 What is the relationship between ProgTutor's adaptive learning paths and student progress (e.g., exercise completion and exam performance) in a high school programming course?

These questions are explored through the design, implementation, and evaluation of ProgTutor, and the results are discussed in Section V. Our findings contribute to the broader discourse on AI-driven personalization in education and how it fosters IA, while highlighting actionable insights for practitioners and researchers.

II. RELATED WORK

To achieve reliable and effective personalized learning, it is necessary to develop adaptive models that create student profiles from long-term experience, and this requires a lot of data obtained from real interactions with the tool [12]. To this end, the current version of ProgTutor has been implemented as an expert system, being the student learning path personalized through a probabilistic rule, like in other successful systems [4]. This approach is fully functional, allows for the required data capture, and provides a different learning path for each student, although no profiling is carried out in this version. As a consequence, the adaptability of ProgTutor is still moderate, but we will refer to it as an ITS throughout this article to use an established concept and to frame what the tool will be in the near future.

A. ITS for Programming Education

In recent years, numerous ITSs have been developed to support programming education, addressing specific challenges faced by students when learning to code. One such system, iSnap, was introduced by Price et al. [13] as an extension of the Snap! programming environment. The findings from this study highlight that iSnap's hints not only helped students complete programming tasks but also fostered a positive reception, with students actively engaging with the hints. Despite these benefits, iSnap primarily focuses on hint generation, and it does not address learning path adaptation, a key aspect of our proposal here.

Building upon this concept, Roldán-Álvarez and Mesa [14] developed another ITS designed to support Python programming

education by providing feedback during the development process, answering programming questions and offering examples. Utilizing a reduced version of the BERT model, TinyBERT, the system is trained with a dataset of 800 questions to respond to a set of 16 distinct programming question categories, each representing a common type of question in Python sessions, offering precise responses tailored to the students' inquiries. However, this ITS is also somewhat limited in scope, as it solely focuses on answering preclassified types of questions and does not evaluate students' code or provide deeper personalized information.

Expanding on the applications of ITS for broader programming languages such as C++ or Java, Sharma and Harkishan [15] introduce an ITS that employs metrics such as syntax error detection, number of attempts, time spent on each task, assessments and scores, and the student's activity history to feed a model that adjusts the difficulty level of tasks based on the student's current knowledge. The feedback provided ranges from error correction to improvement suggestions, access to hints, and complete solutions when necessary. Although this ITS is among the most comprehensive analyzed so far, it lacks a more practical application, as students show greater interest in activities where they can execute their code and obtain more visual results.

A special mention must be done to the potential of learning analytics when applied to programming instruction, as evidenced by Blikstein et al. [16], which analyzes over 154 000 code fragments from 370 students in an introductory programming course. Machine learning techniques were employed to identify coding behavior patterns and predict final grades, revealing significant correlations between coding evolution and academic performance. These findings underscore the potential of process-based evaluation methods to enhance programming education by providing real-time feedback and personalized learning strategies. In line with this approach, Charitsis et al. [17] introduce a tool for capturing and analyzing code snapshots throughout the programming development process, addressing the limitations of traditional assessment methods that only consider final code submissions. The tool visualizes code evolution, detects syntax errors, and highlights complexity, facilitating a deeper understanding of student learning patterns. In addition, its integrated editor enables real-time code experimentation, making it particularly valuable for one-on-one feedback sessions between students and instructors.

In contrast to prior ITSs emphasizing automation, our approach positions the AI as a reckoning partner that amplifies the teacher's and learner's judgment, aligning with IA perspectives in which complementary human-machine roles outperform either alone [11].

B. ITS for Robotics Education

In the specific case of robotics education, we can find many references related to intelligent tutoring robots (ITRs) [18], which build upon the foundations established by ITSs. Unlike traditional ITSs, which primarily operate in virtual environments and often rely on screen-based interactions, ITRs utilize physical robotic agents to facilitate learning through embodied interactions. By leveraging advanced AI techniques,

ITRs offer personalized learning experiences in a more interactive and engaging manner, incorporating elements such as natural language processing, machine learning, and adaptive feedback mechanisms [19]. However, the embodiment of ITRs and their real-time interactions introduces complexities related to safety, usability, and emotional engagement, and exploring these challenges and opportunities lies beyond the immediate scope of this work.

In contrast, using a robotics simulator [20] can effectively mitigate the challenges associated with physical embodiment while enhancing usability in classroom settings. Robotics simulators allow students to engage with virtual robots in a controlled environment, where they can program and interact with these digital agents without the physical risks and logistical constraints of real-world robots. Furthermore, simulators can provide a more consistent and repeatable learning experience. Students can experiment with various programming techniques and see the immediate results of their code in real time, facilitating a deeper understanding of the concepts being taught. In addition, the flexibility of a virtual environment allows for a wider range of scenarios and challenges that might be impractical or unsafe to replicate with physical robots.

While there are several robotics simulators that use programming as a tool, most of them are focused on robotics rather than the underlying programming concepts. Robotics-Academy [21], a remarkable example in this scope, exemplifies this trend, and it primarily targets users with prior programming knowledge, which can alienate beginners who may find the content overwhelming without foundational support. The Gazebo simulator [22], though powerful, can also be complex and resource intensive, requiring substantial computing resources that not all users may possess. Furthermore, despite providing automatic evaluators for immediate feedback, the absence of a structured learning path can leave students unsure of their next steps, potentially diminishing motivation.

Recent advancements in educational data mining and learning analytics have contributed significantly to the development of ITS for robotics education, particularly in real-world classroom environments. One such study by Scaradozzi et al. [23] evaluated 197 high school students in Italy, employing four machine learning techniques to predict student success in educational robotics course. The authors were able to identify three primary problem-solving styles, which can enhance pedagogical assessment and facilitate educational robotics integration into curricula. Similarly, another study by Jormanainen and Sutinen [24] presents an open monitoring environment (OME) based on data mining to support real-time teacher intervention in robotics' classrooms. Given the unpredictable nature of student learning trajectories, the OME enables educators to model and analyze the learning process dynamically, leveraging decision tree algorithms to classify student progress effectively.

C. Research Contribution

In light of these challenges and opportunities, the ProgTutor ITS presented here is designed to fill this niche: *develop a robotic-based simulation framework for teaching programming*

fundamentals that integrates an ITS to deliver a personalized learning experience. Unlike platforms such as CoderZ [25] or CodeCombat [26], which often prioritize gamification and lack formalization in teaching fundamental programming concepts, ProgTutor focuses on foundational skills, organizing the challenges based on incremental programming concepts, starting from the elementary level.

This ITS provides automated evaluations and feedback on coding errors, ensuring that learners receive the support they need to progress effectively. In addition, although in a simple way in the current version, ProgTutor features dynamic learning paths tailored to individual student needs, guiding users through essential programming concepts in a logical and coherent manner. Finally, it must be pointed out that the ProgTutor design is teacher centered, who is intrinsically placed in the decision loop, having the student's progress under control all the time. Supporting and enhancing the teacher's work is a key premise.

III. PROGTUTOR

The ITS presented here has been specifically designed to facilitate the learning of programming in the *Python* language, one of the most popular languages today in both professional and educational environments. ProgTutor uses the educational robot *Robobo* [27] as a core pedagogical tool. This robot consists of a mobile base with sensors and actuators that are combined with a smartphone, which gives it advanced technological capabilities, such as a camera, microphone, speaker, touch screen, and a powerful central processing unit (CPU). Robobo has a Python library and detailed documentation, which is necessary in the scope of this work.

Following the argument explained earlier, in ProgTutor, students do not directly program the physical robot. Instead, they interact with RoboboSim [28], a 3-D simulator developed in Unity that accurately recreates the robot's environment. This simulator acts as the main interface for students to perform their programming practices virtually, and it is paired with the code editor Visual Studio Code (VSC), keeping the focus on developing key skills without the need for access to specialized hardware. In this way, ProgTutor ensures an accessible and efficient learning environment where students can experiment and improve their Python skills in a controlled manner.

A. Conceptual Architecture

ProgTutor follows a classical ITS architecture [29], structured in four main modules, highlighted in violet in Fig. 1: *Interface Module*, *Tutor Module*, *Student Module*, and *Domain Module*. These modules make up the core of the system, managing user interaction, teaching personalization, learner progress tracking, and educational content management.

In addition to these core modules, ProgTutor includes five functional components, represented in black in Fig. 1. These components, called *Adaptation Models*, *Adapted Activity*, *Data Capture*, *Data Analysis*, and *Feedback Results*, provide key functions such as processing data to adjust activities, personalizing learning according to student progress, recording interactions, and giving immediate feedback. Operationally, the Tutor and

TABLE I
PROGRAMMING CONTENTS BY DIFFICULTY LEVEL

Block	Level	Programming Contents	Sensors and actuators
Input/Output	1	Simple input and output without type conversion, using a basic expression (formula) to calculate values. Focus on simple variables.	Simple actuators to move the robot (move wheels by time) and express itself (say text, play sounds or activate LEDs).
	2	Input with type conversion, handling more complex mathematical expressions, and outputting a different data type. Multiple variable types.	
	3	Includes all of level 2 content, but with a slightly more complex algorithm or expression.	
Conditionals	1	Simple conditionals, using <code>if-else</code> or <code>if-elif-else</code> structures. Simple or linked conditions using a single <code>and/or</code> operator.	Simple actuators to move the <i>face</i> of the robot and set different emotions. It also includes sensors to detect the position of the <i>face</i> .
	2	Multiple <code>elif</code> statements. Complex conditions, using multiple <code>and/or</code> operators.	
	3	Nested <code>if-elif-else</code> structures. Increased algorithmic complexity.	
Loops	1	Loops to validate input data and iterate as part of algorithm implementation.	Actuators to move the robot are added (move wheels without time control). The number of sensors is increased to detect more information about the robot (wheels positions, orientation of the robot, identify Blobs).
	2	Loop for validating input data, combined with a conditional statement within the loop to handle the algorithm.	
	3	Loop for validating input data, combined with nested loops or exercises of greater algorithmic complexity.	
Functions	1	Adaptation of level 2 looping exercises. The statement specifies the functions to be implemented, including input and output parameters.	More complex sensors to detect QR and objects are considered.
	2	The statement indicates the number of functions to be performed, but without indicating their parameters.	
	3	The statement does not indicate the number of functions. Still, the student is urged to structure his or her program appropriately and the evaluation will verify that there is a minimum number of functions.	

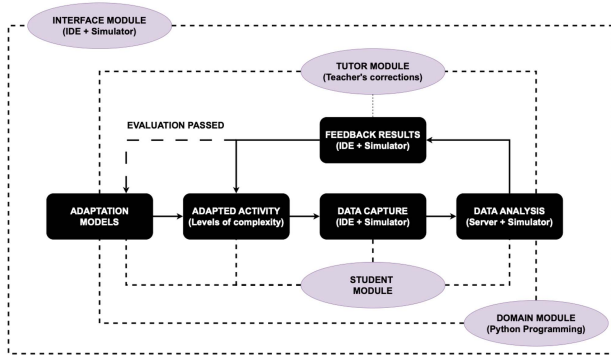


Fig. 1. ProgTutor flowchart and basic building blocks. Dotted lines connect the core modules with their associated functional components.

Data Analysis modules handle reckoning (assessment, sequencing), while the teacher-facing interfaces preserve judgment, enabling timely micro and mesolevel interventions.

Each of the modules will be detailed in the following sections, describing their specific features and functionalities.

1) *Domain Module*: The *Domain Module* encompasses the specific knowledge about Python programming that is intended to be covered by using the tool. Fig. 1 represents that this knowledge is used in the *Adaption Models* block to assign a new exercise to the student adapted to their level, and in the *Data Analysis* block to evaluate the student's solution. Both blocks will be explained in detail later.

In developing our ITS for students, programming and robotics professors at the University of A Coruña, with many years of experience teaching these subjects, have carefully designed exercises that cover, in this version, four fundamental programming topics: input/output, conditionals, loops, and functions. Each of these core topics has been integrated with robotics concepts to enhance practical understanding and application, as detailed in Table I. This approach not only introduces students to

EXERCISE 'Incremental speed'

Make a Python program that does the following steps:

1. Ask the user by keyboard for the maximum speed that Robobo should reach. Check that the speed is greater than 10 and a multiple of 5. If the number entered does not meet these conditions, it must be asked repeatedly until it does (`while` is recommended).
2. Make the robot move in a straight line (use `moveWheels()` method and not `moveWheelsBytime()`) with an initial speed of 10 increasing by 5 until the maximum speed is reached.
3. Pause for 0.5 seconds (using `wait`) each time the speed is increased.
4. Speak (`sayText`) the value of the speed each time the speed is increased.
5. Stop the motors once the maximum speed is reached.

Fig. 2. Example of the proposed exercise: a loop exercise of beginner level.

basic programming principles, but also demonstrates how these principles apply in a realistic use case.

To accommodate different progress rates, each programming topic is further divided into three levels of difficulty: 1) Beginner; 2) Intermediate; and 3) Advanced. Significantly, while each topic introduces new concepts, programming is an incremental task. Therefore, concepts learned in previous topics are incorporated into the challenges of the new one, reinforcing prior knowledge as students take on more advanced tasks (see Table I). Similarly to programming concepts, robot functions are also progressively introduced—ranging from simple methods that involve turning ON an LED or expressing an emotion to more complex sensors that detect a QR code (see last column in Table I). However, it should be recalled that ProgTutor's main aim is not to teach about robotics, but computer programming.

Bloom's revisited taxonomy was employed to classify the educational objectives of the exercises [30]. This taxonomy organizes cognitive processes into a hierarchy of six levels: 1) Remembering; 2) Understanding; 3) Applying; 4) Analyzing; 5) Evaluating; and 6) Creating. The exercises used in this study are classified at Level 5, as the students are given a complex problem, as depicted in Fig. 2, and must develop a solution from

scratch. As can be seen in the statements, the exercises have no limitations regarding the quantity or forms of the functions to be developed, the expected output type, or the code format.

2) *Student Module*: The *Student Module* contains all the student's data stored by the ITS, as a sort of snapshot of their identity and progress, including login credentials, enrolled courses and groups, assigned exercises, source codes submitted, and a record of various interaction metrics. As seen in Fig. 1, we update and use the student's model in the *Adaption Models*, *Adapted Activity*, *Data Capture*, and *Data Analysis* blocks.

During the *Adapted Activity* block execution, the student reads the assigned exercise, develops a program to solve the challenge, and tests the solution. Throughout this process, different metrics are recorded on the *Data Capture* block, which are used to adapt the learning path, provide real-time feedback and hints, and display relevant information to the teacher. These data are also stored to develop the models that will be used in future versions of the tool.

A first set of metrics is captured from the student's interaction with VSC, and they are related to programming issues. A second set is taken from RoboSim, and it includes metrics related to the students' interaction with the simulator and their ability to understand and solve the practical challenge proposed by the system. All metrics are explained as follows:

- 1) total number of programming errors made while solving a challenge;
- 2) number of programming errors, divided into ten different metrics, each representing the most common errors students make when programming (NameError, SyntaxError, ValueError, TypeError, IndexError, IndentationError, KeyError, ImportError, FileNotFoundError, and AttributeError), and an extra metric that stores any errors different from the defined categories;
- 3) number of conceptual doubts solved by the teacher. When a student needs help with an exercise, and the teacher assists them, this is recorded as it provides a measure of the students' autonomy and how it evolves during the course. Each request is directly marked by the teacher in the VSC extension through a specific button;
- 4) total time required by the student to solve the challenge;
- 5) number of times the student submitted the challenge for evaluation and did not pass it;
- 6) number of times the student resets the simulator world where the challenge was being executed;
- 7) number of times the student reads the statement;
- 8) number of times the student tested the challenge without evaluating it.

These metrics are used on the *Data Analysis* block to evaluate the exercise made by the student and by the *Adaptation Models* block to assign them a new exercise.

3) *Tutor Module*: The *Tutor Module* simulates the teacher's experience in a specific domain, in this case Python programming. It encompasses the evaluation criteria for each exercise, the feedback provided on errors in the exercises, and the assignment of the next exercise based on the student's metrics. It includes the *Data Analysis*, *Feedback Results*, and *Adaption Models* blocks in Fig. 1.

TABLE II
PARAMETERS FOR DIFFERENT TOPICS

Topic	1	2	3	4	1	2	3	4
m_d	0	0	0	0	2	2	2	2
m_e	8	10	15	20	8	10	15	20
$1 - \epsilon_t$	0.92	0.93	0.91	0.90	0.52	0.65	0.68	0.70
ϵ_t	0.08	0.07	0.09	0.10	0.48	0.35	0.32	0.30
α_t	0.200	0.141	0.115	0.100	0.200	0.141	0.115	0.100
β_t	0.010	0.007	0.006	0.005	0.010	0.007	0.006	0.005

The *Data Analysis* block is executed once the student submits their solution for evaluation. The code runs in RoboSim, checking that there are no execution errors. While the robot performs its programmed actions, the following checks are carried out, based on the evaluation criteria defined by the teacher, which will be detailed in Section III-B3.

- 1) The code uses all the proposed commands, with the correct values and in the correct order.
- 2) The robot executes the actions within the time allowed.
- 3) The number of times the robot collides with an element in the simulated world is less than indicated.
- 4) The final position of the robot is correct.

When the evaluation ends, the *Feedback Results* block shows what requirements were not met and what errors, if any, occurred. If the evaluation fails, the students stay on the same exercise (*Adapted Activity* block). If all the requirements were met, the evaluation is passed and the *Adaption Models* block assigns a new exercise, based on the student's metrics.

For this first version of ProgTutor, and with the aim of obtaining a proper set of data to train more complex models in next versions, it was decided to implement a simple expert system based on a probabilistic rule to assign the new exercise. The rule uses two metrics: the *number of conceptual doubts solved by the teacher* (m_d) and *number of times the student submitted the challenge for evaluation and did not pass it* (m_e). Once a level $n \in \{1, 2, 3\}$ exercise on topic $t \in \{1, \dots, T\}$ (input/output, conditionals, loops, etc.) is successfully solved, the rule establishes a probability:

- 1) ϵ_t that the system will propose a new exercise of the same level n (at most three times);
- 2) $1 - \epsilon_t$ that the system will propose an exercise of the next level $n + 1$, or go to the next topic $t + 1$ if $n = 3$ being

$$\epsilon_t = \max(0.05, \min(0.95, \alpha_t m_d + \beta_t m_e)) \quad (1)$$

$$\alpha_t = \frac{0.2}{\sqrt{t}}, \quad \beta_t = \frac{0.01}{\sqrt{t}}. \quad (2)$$

An example of how this rule is calculated and how the number of teacher assists (m_d) affects the final result more than the number of evaluation errors (m_e) is shown in Table II. It should be noted that the rule has a certain randomness, as there will always be a probability that the student will remain at the same level.

4) *Interface Module*: The *Interface Module* is responsible for managing the interaction between the student and the tutoring system, and it encompasses most of the other modules and blocks, as displayed in Fig. 1. The ProgTutor interface is made up of two independent tools: the VSC IDE, where students write

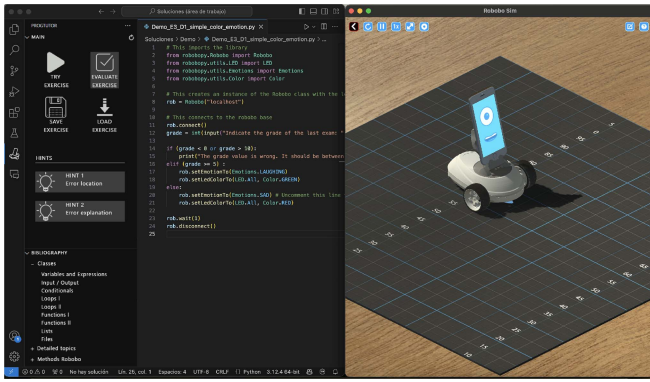


Fig. 3. Left: VSC extension for ProgTutor. Right: ProgTutorSim.

their programs, and the ProgTutorSim, an application with a 3-D simulator in which they can test them, as shown in Fig. 3.

An specific extension was developed for VSC, which includes the following buttons and functionalities.

- 1) *Try Exercise*: It executes the currently open Python program in VSC, checking syntax and runtime errors.
- 2) *Evaluate Exercise*: It uploads the code to the remote server and calls ProgTutorSim for its evaluation. The button is initially disabled, and it is only enabled when the program code has been executed at least once without any syntax errors. If any changes are made to the code, the button is disabled again and only re-enabled when the code is tested again without errors.
- 3) *Save Exercise*: It uploads the code being developed for the current challenge to the ProgTutor database.
- 4) *Load Exercise*: It downloads the last saved program from the ProgTutor database into the VSC text editor.
- 5) *Hint Buttons 1 and 2*: The first hint highlights in red the line where a programming error occurred, and the second hint provides an explanation of the type of error that occurred. These explanations were previously generated using the GPT-4 model through a direct instruction prompt combined with a structured format. The most typical Python errors were included in the prompt, which explicitly requested an error-specific explanation on each case, formatted clearly for the student age. The obtained explanations were revised and included in the ProgTutor database. Initially, these buttons are disabled, and the hints are always shown within the code because it is considered that when the student starts the course, they have no programming knowledge. As the course progresses, the hints are removed from the code, and the buttons are enabled so that the student can request them on demand. A hint example is shown in Fig. 4.
- 6) *Bibliography*: It shows a list of the programming topics covered in class and extra help. Each bibliography topic links to an external resource, where the Jupyter notebooks, used as theoretical references and explained in Section IV, can be accessed by the student. There is also a link to all Robobo documentation and an explanation with examples of each topic covered throughout the course. A documentation example is shown in Fig. 5.

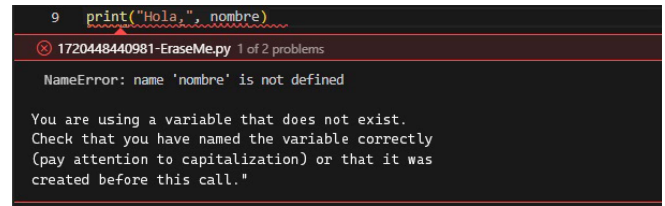


Fig. 4. Hint: at the bottom a text is shown, in natural language, that attempts to help the student solve the type of syntactic error detected.

Structure and concepts to remember

```
if Logical condition :
    # Execute this code if the condition is true (True)
else:
    # Execute this code if the condition is false(False)

if Logical condition 1:
    # Execute this code if condition 1 is true
elif Logical condition 2:
    # Execute this code if condition 1 is false and condition 2 is true.
else:
    # Execute this code if neither condition 1 nor condition 2 are true
```

Remember

- To indent the source code correctly
- Always handle all possible error situations caused by user inputs, arithmetic operations, etc.
- Comparison operators: ==, !=, <, <=, >, >=
- Boolean operations: and, or, not

This table shows the result of applying these boolean operators to two variables:

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Fig. 5. Example of Jupyter notebook available in the bibliography section.



Fig. 6. Main menu of ProgTutorSim. The top button is used to enter into the exercises' section.

The ProgTutorSim application (see Fig. 6) serves as the primary element to enhance student attention and motivation, using all functionalities and flexibility typical of video game interfaces [27]. The core functionality of the simulator is on the ProgTutor submenu, displayed in Fig. 7, where the student can access its currently assigned exercise. Here, the information of the exercise is broken down into the following four sections.

- 1) *Statement*: It shows the statement of the exercise, explaining how the challenge should be solved (see Fig. 7, top).

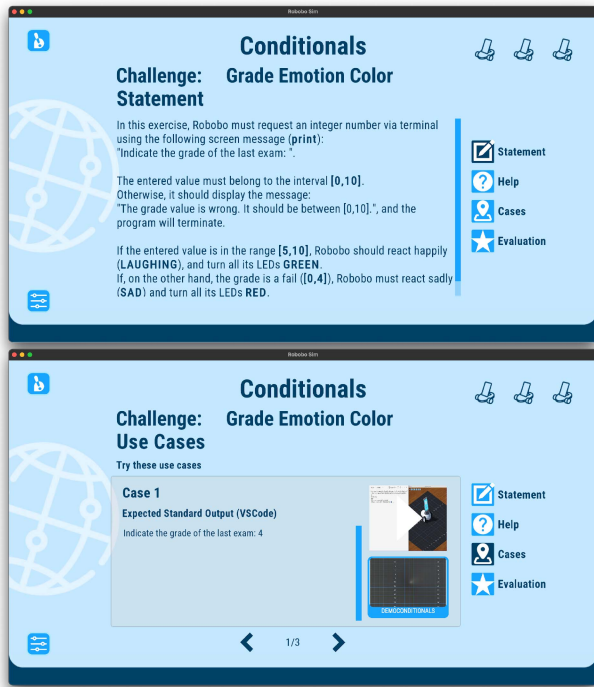


Fig. 7. Top: exercise statement, explaining the goal of the program to develop. Bottom: use cases provided to test the solution.



Fig. 8. Top: example of a world in which the program must run. Bottom: evaluation summary with the error explanation.

- 2) *Help*: It shows the Robobo methods required in the challenge.
- 3) *Cases*: It shows the expected output of the program, both in text and video form, for each of the use cases of the exercise (see Fig. 7, bottom). It also serves as an entry point to the in-game world where the exercise is to be tested (see Fig. 8, top).
- 4) *Evaluation*: Once the students' code is submitted for evaluation, any errors made are displayed here as feedback (see Fig. 8, bottom). The provided feedback does not include details about the student's code or the possible solutions, to exercise judgment rather than only reckoning.

B. Software Architecture

To create the ProgTutor system, the client–server architecture shown in Fig. 9 was developed. Users log in with a unique username and password (anonymized), and upon successful validation, they receive a distinct identification token.

The following sections describe the main elements of the software architecture.

1) *Client*: The client features a tailored version of RoboboSim equipped with an internal application programming interface (API) based on the representational state transfer (REST) architecture, facilitating communication between local components and the simulator while allowing access to user identification tokens. In addition, the client employs the VSC IDE with our custom extension designed to interact with both the server and the simulator.

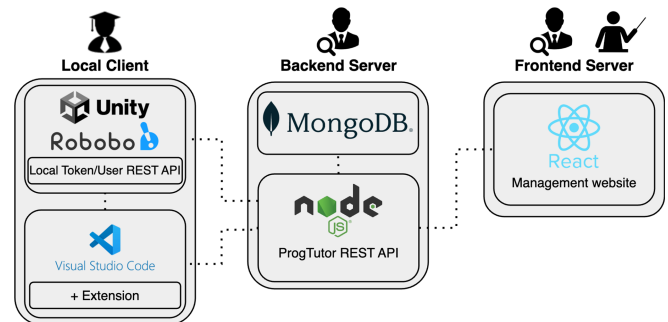


Fig. 9. ProgTutor software architecture diagram.

2) *Database*: On the server side, we set up a NonSQL database using MongoDB to store all relevant system information. To enable database interactions, we created a Node.js REST API that structures MongoDB data using Mongoose and provides secure Create, Read, Update, and Delete methods over HTTPS. Both the simulator and the VSC extension interact with MongoDB data through this interface.

As seen in Fig. 10, the database contains details about available courses, organized into thematic blocks (Python topics) with exercises that students must complete. Each exercise comprises a statement and three distinct use cases that illustrate how to run the program with different input data; these use cases are also utilized for automated assessment. Each use case specifies a unique in-game world within RoboboSim where the program must operate, along with a video or image depicting the expected output. Moreover, we keep records of the exercises assigned to each student along with their corresponding performance metrics.

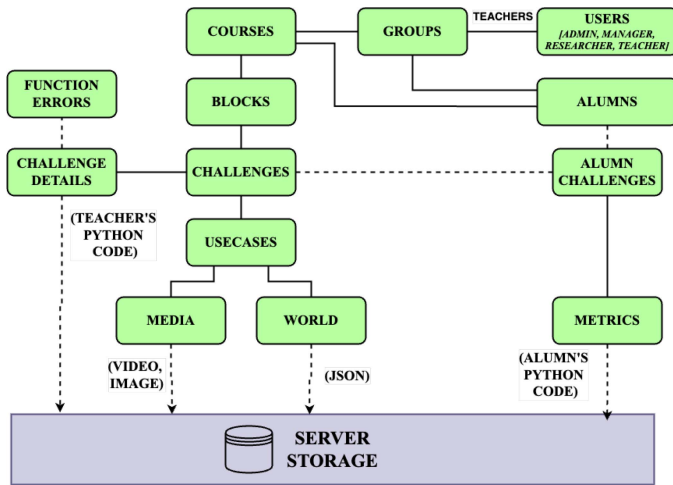


Fig. 10. Database diagram.

The database also keeps track of login information for students and various user roles, including administrators, managers, researchers, and teachers. The students are enrolled in a course and are organized into groups, each with at least one teacher assigned. This structure enables us to filter the data so that teachers can access only the information related to the students in the groups they oversee.

More complex data, such as the illustrative videos and images, the student's code, and JSON files containing world data are stored in the server's storage, with references to them maintained in the database.

3) *Management Website*: To create the exercises used during the course and store them in the database, a web form with a simple and user-friendly graphical interface was developed in React, making it easy for all users to navigate. This form allows for the input of all the data required for the specific challenge, including general information about the exercise shown to the student, such as the name, description, examples, and a video of the use cases. The teacher who designs the exercise defines the specific parameters they want the automated system to evaluate. These parameters can vary, allowing for a dynamic evaluation process. Once the exercise is created and saved, the teacher has access to the full list of exercises, where they can modify or delete them as needed, maintaining complete control over the tool at all times. Thus, the system makes probabilistic reckoning decisions about what to assign next, while teachers retain authority to override and guide based on professional judgment.

To enable the teacher to monitor student progress throughout the course, a dashboard was developed in the management website where they can access all available information. This information is presented in a manner that facilitates informed decision making regarding the actions to be taken with each student, allowing the teacher to focus attention on cases that require it. In the screen shown in Fig. 11, the teacher selects the group they are teaching from a list of groups and automatically obtains all relevant information. The display includes the number of exercises each student has completed, categorized by topic and level. It also shows the percentage distribution of students by study topics, the average time each student takes to



Fig. 11. Main screen of the teacher's dashboard.

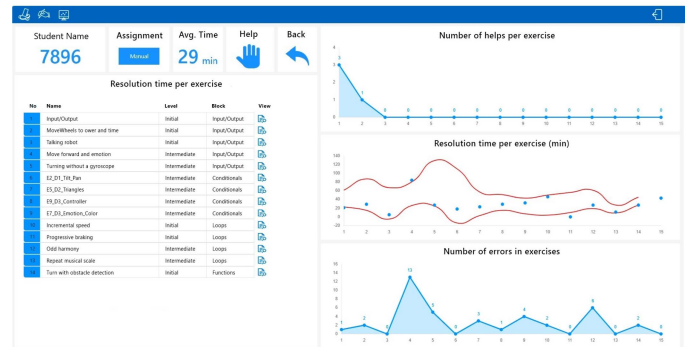


Fig. 12. Student progress screen.

solve different challenges, and the average number of queries per student. In addition, there is a graph that shows the average number of times each student submitted the assigned challenges for evaluation.

In IA terms, the dashboard aggregates reckoning (metrics and trends) to support teacher judgment about microlevel supports (an individual student) and mesolevel patterns (a group's pacing). The manual assignment control intentionally keeps educators in the decision loop.

Fig. 12 presents the individual screen showing the available data for each student. Here, the teacher can see the list of challenges completed by the student, along with graphs displaying the number of helps, resolution time, and the number of errors for each challenge. This screen also includes interactive buttons: the help button allows the teacher to increase the help metric for the exercise the student is currently working on, and the assignment button gives the teacher the option to manually assign an exercise to the student, thereby retaining control over the tool and ensuring that the teacher's decisions take precedence over those of the AI models.

The final available screen, shown in Fig. 13, appears when the teacher clicks on any exercise completed by the student. In this window, the solution code for the selected challenge is displayed, along with the last solution submitted by the student for evaluation. This window allows the teacher to compare both codes, enabling them to provide more prompt and detailed feedback to the student when necessary.

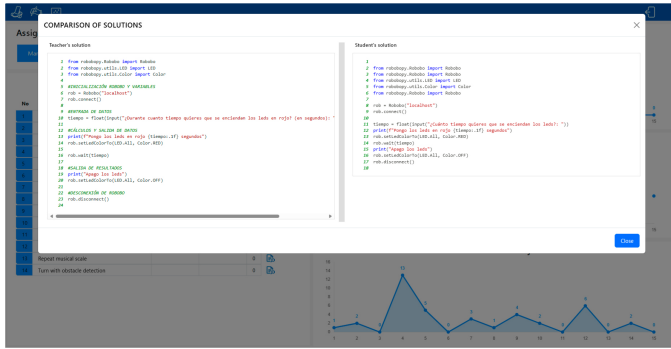


Fig. 13. Window that allows us to view the source code of the teacher's solution and that of the student.

IV. METHODOLOGY

A teaching methodology for the use of ProgTutor in a classroom is proposed here. It is designed to be flexible and adaptable to different learning environments and student needs, but it has been adapted from our experience during a year course in an official subject. This methodology emphasizes a blended learning approach, combining self-paced learning facilitated by the ITS with structured teacher-led instruction.

The current methodology has been designed for upper secondary school students (ages 16–20) following a Python fundamentals course. While no prior technical expertise is mandatory, a basic understanding of digital literacy and mathematical concepts is beneficial. It is particularly well suited for students interested in technology and Science, Technology, Engineering, and Mathematics fields, as ProgTutor is based on a practical approach through robotic challenges. It aims to foster computational thinking skills in students more than just programming, in the sense that it involves a set of strategies and techniques designed to understand and solve complex problems, instead of focusing on the specific instructions required to implementing these strategies in a computer language [31].

A. Classroom Preparation and Technical Requirements

Adequate classroom preparation is essential to ensure a smooth and effective learning experience. Ideally, there should be one computer per student, which could be either a desktop or a laptop, but in both cases, an external mouse is essential for a positive user experience, as touchpads are unsuitable for handling the simulator. A projector or large screen is also necessary for teacher-led presentations and demonstrations. In addition, a stable internet connection is required to access online resources.

Regarding the technical specifications of the computers, it is crucial to ensure that all computers meet the minimum requirements to run the software properly. A monitor with a suitable resolution is essential for clearly visualizing the simulator and code editor. In addition, the computers must have the necessary software installed. The requirements are shown in Table III.

The current ProgTutor version does not rely on machine learning models that could imply ethical or legal considerations in the scope of the AI Act. It captures data on student performance

TABLE III
COMPUTER REQUIREMENTS

Hardware	Operating System	Software Application
4–8 GB RAM	Ms Windows 7 or higher	Python 3
Intel i3 9300 or better (and AMD equivalents)	MacOS 11 or higher	Visual Studio Code
0.5–1 GB disk space	Debian 10 (and equivalent distributions)	VSC Jupyter extension
Resolution: 1920×1080		RoboboPy library
Mouse		ProgTutor_vsc_plugin
		ProgTutorSim

Internet connection is required. A dedicated graphics card is not required but is recommended. Otherwise, a CPU with an integrated graphics chip is recommended.

when solving an exercise to decide the next exercise to pose to the student. Then, they are learning progress data, not bio-metric, but it could be considered that ProgTutor is profiling the students, so parental consent is required before starting the sessions. The ethical committee of the school should verify such forms, which should be passed to all students beforehand.

Finally, it must be highlighted that a set of Jupyter notebooks and simple Python exercises for every topic established in Section III-A1 are shared with the teachers at the beginning of the course. They can use their materials, but it is recommended to review the ones proposed here, as they follow the same complexity level as the ProgTutor ones.

B. Session Structure and Instructional Design

The methodology outlines a structured approach to guide the learning process, incorporating self-paced and teacher-led components. The course is scheduled in 50-min sessions, as this is the duration of classes in secondary schools in the region of Spain (Galicia) in which the tool was tested, but this can be adjusted to fit the specific context of other educational institution. Similarly, the number of sessions allocated to each of the topics explained in Section III-A1 can vary, and the teachers can modify them based on their considerations. We have proposed a range of sessions from 14 to 20, depending on the students' pace.

The proposed session schedule is shown in Fig. 14. The first two sessions (left part) are preparatory, to introduce the students to a basic knowledge of the Python programming language and the tools they will use during the course. Specifically, the first one focuses on familiarizing students with the technological tools. It is mainly practical, and the teacher should explain the following contents:

- 1) the configuration and basic use of VSC;
- 2) Jupyter Notebooks as a platform for interactive coding;
- 3) a brief overview of Python fundamentals through an introductory notebook, used also to test the VSC;
- 4) explaining the Python available documentation.

The second session introduces students to the RoboboSim simulator and the available simulation worlds. They learn also how to program the robot in Python using the robobo.py library, following a step-by-step guidance of its fundamentals. An explanation of the robobo.py documentation is provided.

The next sessions follow a cyclical pattern with five main goals: topic lecture, Python exercises without the simulator, Python exercises with the simulator, sessions with the ProgTutor

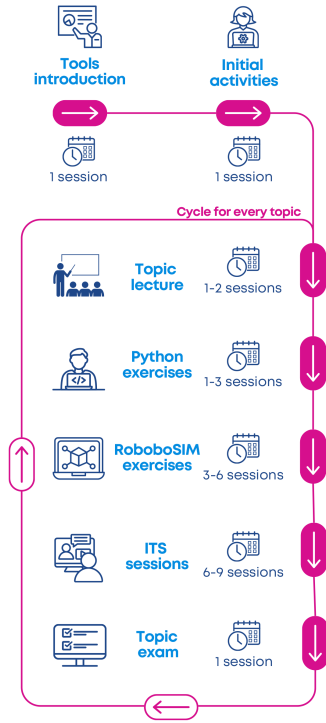


Fig. 14. Proposed session schedule in the ProgTutor methodology.

TABLE IV
SESSION SCHEDULE

Topic lecture sessions per topic	
Topic	Sessions
E/S	1
Conditionals	1
Loops	2
Functions	1

Python practical sessions per topic	
Topic	Sessions
E/S	1
Conditionals	2
Loops	3
Functions	3

Minimum and maximum ITS sessions per topic	
Topic	Min - Max sessions
E/S	3-6
Conditionals	3-6
Loops	6-9
Functions	6-9

ITS, and an exam. The cycle starts with a lecture where the teacher explains one of the programming topics established in Section III-A1, by means of a Jupyter notebook. Depending on the complexity of the topic, this explanation can be performed in one or more sessions. A tentative proposal of the number of sessions per topic is shown in Table IV.

After the topic introduction, some practical sessions should be performed, focusing on basic Python programming with the new topic. The recommended number of practical sessions per topic is shown in Table IV. They do not use the simulator, as it has been observed that it is beneficial for students to spend an initial period practicing the topic fundamentals in basic programs. In

fact, we recommend using simple Terminal or Console output for these sessions. In addition, we recommend that the teacher guides such exercise development, providing students with the final Python solution, so all students have a template to follow.

In the following session, the teacher solves, step by step, one or more exercises with the RoboboSim simulator in the classroom. This session is not a lecture; the students are expected to solve the exercises following the teacher's instructions during the session. It is required because, as explained in Section III-A1, every Python topic involves new robobo.py methods, and it is important to explain them beforehand to avoid students spending much time against robotics issues.

The final part of the cycle consists of several sessions where the students develop the exercises proposed by the ProgTutor ITS. During these sessions, the teacher becomes a facilitator, providing guidance, answering questions, and solving technical issues. It must be pointed out that the first time a student uses ProgTutor, an explanation should be provided about the user interface.

All the students start with the same initial level in the ITS, which corresponds with the first topic explained in Section III-A1 and exercise level 1. However, the ITS allows the students to progress at their own pace, which complicates the normal course of a class, mainly synchronous. In order to avoid enormous differences between students, a minimum and maximum number of sessions is proposed to work with ProgTutor, which are shown in Table IV. Considering them, a simple procedure based on two rules has been defined to advance to the next topic.

- 1) a) When two-thirds of the class moves to the next topic (passing the three levels) and b) the number of sessions is higher than the minimum, the last stage shown in Fig. 14 is activated, and all the students must perform an exam.
- 2) The students that pass to the next level before these two conditions are met are recommended to autonomously study the Jupyter notebook of the following topic and solve simple console exercises on this new topic.
- 3) When the maximum number of sessions is reached but less than two-thirds of students moved to the next level, we recommend the teacher provide more support to the classroom until this threshold is achieved. This could take one or two additional sessions, depending on the number of students that need help.

As commented earlier, when the two conditions to move to the next topic are reached, the next session will be dedicated to assessing the students with an exam about the specific topic. Even those students who did not finish the three levels should go to this exam, as in traditional synchronous teaching, to avoid too much dispersion. It is recommended that the exam consists of Python exercises to develop autonomously and without the simulator, as the project's goal is for the students to learn how to program in Python in a general way, not only with robotics challenges. After this exam, the cycle is restarted and the next topic lecture should be carried out.

An essential part of this methodology is monitoring the students' progress, which is crucial for identifying those falling behind or encountering challenges. The teacher can utilize

the dashboard provided by ProgTutor to track individual and class-wide performance. This information allows the teacher to provide targeted support to students who require additional assistance, ensuring that no one is left behind.

V. VALIDATION AND DISCUSSION

The validation of the first version of the ITS ProgTutor, as well as the first data collection, was carried out at the Sofia Casanova Secondary Education Institute in Ferrol (Spain) during the academic year 2023–2024. A total of 73 first-year high school students enrolled in an Information and Communication Technology course took part in the experience. They were all between 15 and 18 years old (mean age: 15.95, standard deviation: 0.53). In addition, 34.33% were female and 64.18% were male. The study was organized into two groups: a group of 37 students using ProgTutor and its associated methodology, and a control group of 36 students following a traditional methodology. The control group follows a traditional scheme for introductory programming courses, relying on theoretical explanations by teachers, teacher assistance in problem solving, and the use of exercises with data output through a terminal.

To evaluate the students' experience, different questionnaires were provided to them. Section V-A presents an analysis of student responses to the questionnaires provided throughout the sessions, while Section V-B focuses on the academic outcomes and the student learning process. To structure the validation of our proposal, we revisit the research questions presented in Section I, linking each one to specific analyses and results presented below. This mapping allows us to provide a focused discussion around each research objective, grounded in both qualitative and quantitative findings.

- 1) RQ1, concerning student engagement and motivation, is addressed in Section V-A, where we analyze satisfaction questionnaires.
- 2) RQ2, related to teacher support, is discussed in both Sections V-A and V-B, focused on teacher interviews and the use of the dashboard as a monitoring tool.
- 3) RQ3, which explores the impact of adaptive learning paths on student progress, is examined in Section V-B, through quantitative data such as exercise completion, error rates, and help requests.

Although the purpose of the questionnaires has been specified, it is important to detail the specific constructs to be measured, along with the items/questions used for measurement, types of responses, etc. These constructs are the attributes or unobserved behaviors that are the target of the study. This information is provided as follows.

- 1) The *interests and background* of the students are assessed through three dichotomous response items (i.e., Yes/No).
- 2) *Overall satisfaction with the Python programming experience* is measured for both groups using a five-point Likert-type response scale.
- 3) Students' *interest in an approach based on applied programming rather than basic console programming*: In both groups, although especially in ProgTutor, it is interesting to know the evolution of the answers over time

TABLE V
QUESTIONNAIRE 0. RESPONSES ON STUDENT'S INTERESTS

Question	ProgTutor Group (% positive)	Control Group (% positive)
Do you have a high or very high level of motivation to learn in this subject?	68	43
Are you interested in learning how to program computers, cell phones, etc.?	51	37
Have you ever programmed a computer?	22	10

with the use of the tool. This item allows for three possible answers (something more applied, both options equally or more basic).

- 4) A set of items is used to evaluate the different *functional aspects of the tool*. All of these have three response options (Yes/No/NR).
- 5) *Overall satisfaction with the ProgTutor tool* through three dichotomous response items (i.e. Yes/No).

Detailed information on each construct and statistical instruments used in this analysis is provided in the following sections. First of all, an initial analysis was conducted to assess the assumption of normality for all data samples. The Shapiro–Wilk test was performed on each sample, and the results consistently indicated a significant deviation from a normal distribution ($p < 0.002$ for all tests). Given the violation of the normality assumption, subsequent statistical analyses were performed using nonparametric tests to ensure the validity of the findings. All statistical analyses were performed using the Pingouin and SciPy Python libraries.

A. Satisfaction Scores

As mentioned earlier, to evaluate students' perceptions of the course and their experience with ProgTutor, a series of questionnaires was provided throughout the work sessions:

- 1) *questionnaire 0*: before starting Python programming, to assess students' motivation and prior knowledge;
- 2) *questionnaire 1*: at the beginning of the course, to capture early impressions and expectations;
- 3) *final questionnaire*: at the end of the course, to measure overall satisfaction (questionnaire 4 for the ProgTutor group, and questionnaire 2 for the control group).

In addition, the ProgTutor group completed two mid-course questionnaires (questionnaires 2 and 3) to gather impressions on the functionalities and weaknesses of the tool.

Table V shows the results for the three dichotomous response items included in questionnaire 0. As can be seen, the responses obtained show a difference in the initial level of motivation and interest in the subject, which may be related to the number of students in the ProgTutor group who had some previous exposure to programming (22% versus 10%).

Starting with the ProgTutor group, it must be noted that the progress of the students was not homogeneous as it was determined by the dynamics of the ITS itself. For this reason, we are interested in gathering information about how the students

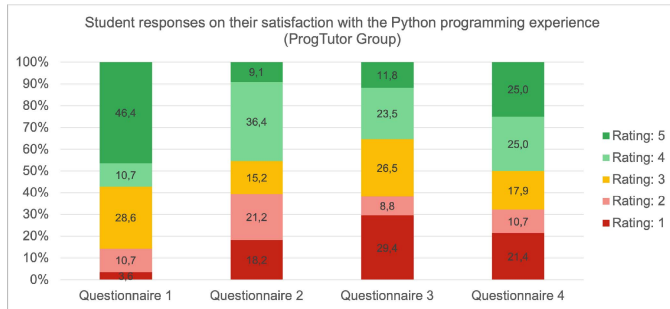


Fig. 15. Student responses (ProgTutor group) on their satisfaction with the Python programming experience. 1–5 rating scale from 1, the lowest rating, to 5, the highest rating.

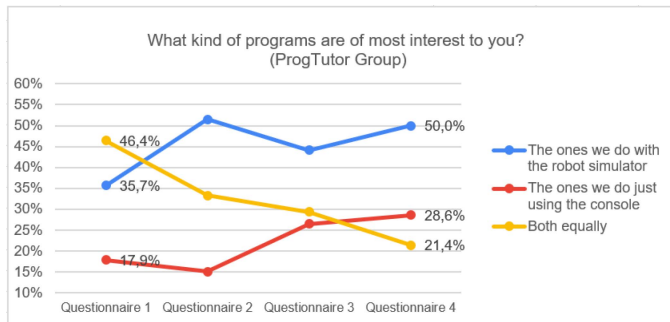


Fig. 16. Student responses (ProgTutor group) regarding their preference for the type of programs to work on.

interact with the ITS on a day-to-day basis in their programming classes. Two aspects of particular interest about which we would like to know the students' opinions along the experience are indicated as follows.

- 1) Are they finding the experience of learning to program in Python satisfactory?
- 2) Do they value positively the ability to solve seemingly more complex problems with richer interaction using the robot simulator, compared to programming more abstract algorithms that involve only simple interaction through a text terminal?

In Figs. 15 and 16, we can see the results obtained for both items. Regarding the first question, Fig. 15 shows that the highest ratings (ratings 4 and 5) correspond to the first monitoring, decreasing in the intermediate questionnaires and increasing in the final evaluation. In this regard, a statistical analysis, using nonparametric Wilcoxon test for paired samples, was performed in order to compare the responses from the last two questionnaires, questionnaires 3 and 4. Results indicate a significant increase in student satisfaction in questionnaire 4 ($Mdn = 3.0$, $SD = 1.41$) compared to questionnaire 3 ($Mdn = 3.5$, $interquartile\ range\ (IQR) = 1.0$), $W = 13.50$, $p = 0.04$, 95%, and $r = 0.65$. The analysis of the IQR on the difference between the two samples indicates a consistent and positive change in the scores given by students between questionnaires 3 and 4. Likewise, the effect size (r) can be interpreted as a large positive effect, indicating a strong tendency for scores to increase from questionnaire 3 to questionnaire 4. These intermediate results are associated with various errors in the functioning of the tool

TABLE VI
INTERMEDIATE RESPONSES TO THE PROG TUTOR GROUP SUPERVISION ON STUDENT'S INTERESTS

Question	Questionnaire 2 (%Yes/No/NR ^{*1})	Questionnaire 3 (%Yes/No/NR)
Do you like the fact that the simulator proposes different exercises to each student?	42/27/30	35/35/29
Are you finding it difficult to learn programming with ProgTutor?	39/61/_ ^{*2}	47/53/_ ^{*2}
Do you find the help provided by ProgTutor is useful when you have an error in the program?	45/36/18	47/47/6
Do you feel comfortable with ProgTutor automatically grading your exercises?	52/27/21 ^{*3}	56/24/21 ^{*3}
Are you comfortable with ProgTutor leveling your exercises?	58/3/39	41/3/56

^{*1}NR corresponds to No Response.

^{*2}Only two possible answers (Yes or No).

^{*3}Third possible answer: "No, I don't know how ITS corrects me."

(which were resolved during the course of the project), as well as improvements both in terms of help, presentation of exercises, etc., and in terms of the functionalities of the tool. This is reflected in the fact that as students progress in their use of the simulator, the percentage of students giving the highest marks increases (questionnaire 4).

For the second question, Fig. 16 shows that only 35.7% preferred the programs with ProgTutor at the first follow-up. However, as students use the tool throughout the course, this assessment improves to 50%. The percentage of students who are indifferent is high at the beginning and decreases as they become more involved with the tool.

As commented previously, two extra follow-up questionnaires were passed to ProgTutor group (questionnaires 2 and 3), with the questions shown in Table VI. From their answers, the following conclusions can be drawn.

- 1) There is a relevant percentage who say that they do not like that the simulator proposes different exercises to each student. The reason given is that they would like to be able to consult and help each other. Although this could limit collaboration, the ProgTutor learning approach is individual, so no peer-programming method was considered.
- 2) A high percentage of students indicate that they do not find difficulties to learn to program with ProgTutor (61% and 53% in the second and third questionnaires, respectively). This difference between the questionnaires is partly to be expected, as the complexity of the subject matter increases as the course progresses.
- 3) In general, students find the help provided by ProgTutor useful when there are errors in their programs. However, in the third questionnaire 47% indicated it was not useful, which seems to be related with increasing complexity of the exercises and Robobo methods.
- 4) More than half of students (52% and 56%, respectively) value positively that ProgTutor corrects their exercises automatically. Even so, it should be noted that, for both

TABLE VII
RESPONSES TO THE OVERALL EXPERIENCE WITH THE PROGUTOR TOOL

Question	Questionnaire 4 (% positive)
Do you think ProgTutor gave you exercises appropriate to your progress?	79
Would you use ProgTutor next year to continue learning to program?	75
Would you find it useful to have more information about your progress?	64

follow-up questionnaires, 21% indicate that they do not understand how their program was evaluated.

- 5) On the question of how they feel about ProgTutor assigning a level to their exercises, the assessment was good (58% and 41%). However, the feedback received from some students shows that this leveling could make them feel inferior to their peers.

Finally, Table VII shows the responses to the three dichotomous response items included in questionnaire 4 regarding overall feedback on the experience with the ProgTutor tool. As can be seen, the results show a considerable global satisfaction with the project. A total of 79% of students agree with the exercises suggested to them by ProgTutor, and 75% of the students indicated that they would use ProgTutor in the next academic year to continue their education. This percentage is divided into 54% of students who clearly state that they find the tool useful, while the remaining 21% make their answer conditional on the bugs found in the tool being fixed. Finally, note that 64% would like ProgTutor to provide more information about their progress. A consistency study, using the Cronbach's alpha measure, signals that these three items are closely related and, to a certain extent, measure the same underlying construct, $\alpha = 0.73$, 95% CI = [0.49, 0.87].

To conclude the analysis of the ProgTutor group's satisfaction with the experience, two issues deserve special attention.

- 1) As mentioned in the analysis of Figs. 15 and 16 (questions included in the four follow-up questionnaires) and Table VI (items common to questionnaires 2 and 3), there are fluctuations in the scores obtained for the same satisfaction items in the ongoing questionnaires. The main reason for this was some technical problems found during the sessions' development that had to be solved during the course. Despite these issues, the overall satisfaction of the students was high, as it can be observed in the rating of the final questionnaire.
- 2) With regard to the teachers, in-person interviews were carried out at the end of the course. It was highlighted that they had no previous experience on Python teaching, but they were able to successfully complete the course thanks to the tool and the support provided by the researchers involved in the project. Their overall evaluation indicated that the dashboard information was particularly interesting, mainly to know the exercises that each of their students could finalize (number of exercises per topic and their level), and also the time that the student spent solving the different problems.

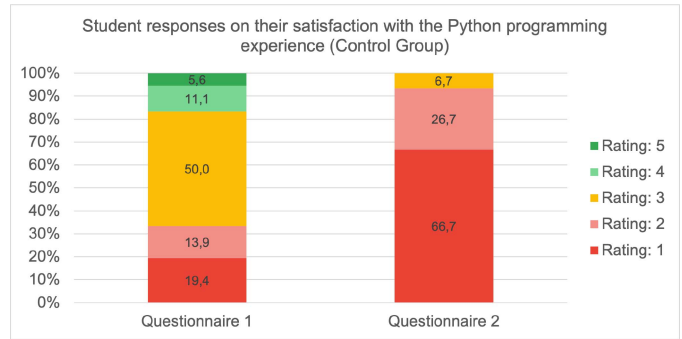


Fig. 17. Student responses (control group) on their satisfaction with the Python programming experience. 1–5 rating scale from 1, the lowest rating, to 5, the highest rating.

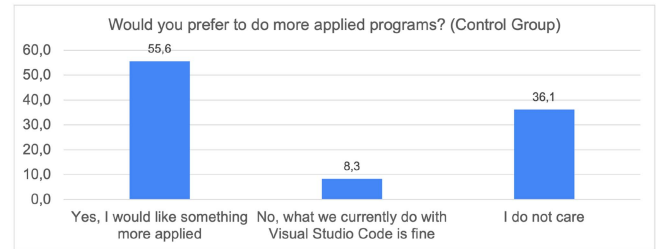


Fig. 18. Student responses (control group) regarding their preference for the type of programs to work on.

Regarding the control group, from the responses received to the questionnaires (see Fig. 17), it is noticeable that a high percentage of students had little interest or no enjoyment in programming in Python. This is clear from questionnaire 2, where no student gives a clearly positive assessment of the experience. At this point, we can compare the overall students' satisfaction level with the python programming experience at the end of the course for the control group (questionnaire 2 in Fig. 17) and for the ProgTutor group (questionnaire 4 in Fig. 15). In this regard, a statistical analysis using nonparametric Mann–Whitney U test for unpaired samples reveals that ProgTutor group shows significantly higher satisfaction ($Mdn = 1.0$, $IRQ = 1.0$) than the control group ($Mdn = 3.5$, $IRQ = 2.25$), $U = 348.5$, $p < 0.001$, 95%. In regard to the effect size ($r = 0.66$), its value can also be interpreted as a large positive effect, indicating a strong tendency for scores to be higher in the ProgTutor group.

On the other hand, as shown in Fig. 18, 55.6% point out that these students would prefer to do more applied programming (such as a smartphone app, a robot, a video game, etc.). This assessment is in line with the opinions of the students in the ProgTutor group in the first weeks of the course, which reinforces the practical approach proposed with this ITS.

B. Learning Achievements

Despite the fact that Python programming was a new content and its implementation was an experimental activity in the school, the ProgTutor group covered most of the planned content (except for the functions topic which was not completed by all the students). Thus, at the end of the experience, a total of 32 students were assessed in the first three topics (input/output,

conditionals, and loops) according to the methodology explained above (topic exam). Of the students who completed the exams, 83.33% passed the assessment, and of these, 33.33% achieved the highest mark. These results show that the programming learning goals were met, the students were progressing normally, and the objective assessment was considered to be within the normal range.

When analyzing the information collected on the students' learning process with the ITS, it is observed that some metrics provide conclusive results, as discussed in the following.

- 1) *Number of exercises completed*: As expected, the higher the number of exercises completed, the higher the grade obtained in the exams. The students with the highest marks completed 11 exercises (median) in ITS (there are occasional situations where some students complete few exercises but make good progress in their learning, as those with prior knowledge), while the median for failing students is 7.5 exercises. Hence, the ITS allows different rates of progress according to the student's needs to achieve the learning objectives.
- 2) *Total number of programming errors made while solving the challenges*: Students with a higher number of exercises made more errors, but if we take into account the weighting, this number is reduced. Students who achieved the highest marks made an average of 1.8 errors, while the others made 3.6. Therefore, in a weighted way, this metric can be used by teachers to identify students with difficulties in their learning process.
- 3) *Number of times the student reads the challenge statement and the help*: The students with the highest mark consulted the statement (6.5 times on average) and the help (1.25 times on average) more often. It is significant that students with lower grades never check the statement or consult help. Again, this is a warning signal for monitoring students with more needs.

Finally, we would like to highlight two important points especially relevant in the scope of IA for teachers and students. On the one hand, ProgTutor allows students to effectively progress in terms of learning objectives, taking into account their specific progress rate, and on the other hand, teachers have objective information about learning pace due to the analysis of the collected metrics. This allows them to focus on the students having more difficulties at each stage of the learning process. This is an important advantage, as the lack of interaction between students of this age and teachers makes it difficult to identify such problems. This is an example of IA in practice: the system's reckoning surfaced who needed help, and teachers then exercised judgment to intervene efficiently.

For the control group, only the first two topics were assessed (input/output and conditionals). Of the students who completed the exams, 88.89% passed the assessment, none achieved the highest mark, and 33.33% achieved the second-highest mark. Again, this was the first time that the teachers faced Python programming teaching, but now they were not able to cover the loops topic in this group. This result reinforces the previous idea of a higher teacher support when using ProgTutor, but it must be confirmed with future research.

VI. CONCLUSION AND FUTURE WORK

As commented in Section I, the goal of reaching effective personalized learning by means of AI has a long tradition in the scope of AIED, but there is still a scarce number of feasible systems that can be used in formal setups. To face this complex challenge, the first version of the ProgTutor ITS has been presented here, together with the initial results of its application during one academic course in an official subject. Details about the ProgTutor conceptual and architectural design have been provided, with a special focus on the educational aspects of the tool, like the specific Python topics and levels that have been selected, and also the teaching methodology that was developed and tested to introduce it in a real classroom. Furthermore, the research questions introduced in Section I have been addressed throughout the validation, providing insights into both student learning outcomes and teacher support, and how they foster IA.

From the student's perspective, the first steps with ProgTutor were complicated, as they were not used to have a learning tool that controlled their individual advancement, so different exercises could be faced by different students at the same level. In addition, they found challenging to cope with the errors and recommendations of the tool, as they were used to receiving a higher degree of guidance and support from the teacher. However, once they got used to this new methodology, most of the students progressed autonomously and appropriately according to the objective Python exams.

Regarding the teacher's view, they realized that this kind of ITS highlights those students with a lower learning rate. Thus, the students with a higher interest in the topic, or with a higher capacity for programming, were capable of advancing autonomously just with ProgTutor's feedback and the documentation included in it, thanks to the adjustment of the exercise level that is the core of this type of system. Therefore, the students progress at their own pace and do not get frustrated at a level that is not adequate for them. However, as a consequence, students with lower interests or skills are clearly identified by the system, and the teacher can focus on them. To this end, the captured metrics and the dashboard have been pointed out as really important for the teachers to enhance their understanding of the learning progress.

While the current work has shown promising results and relevant insights, it must be pointed out that the validation study was conducted in a single high school, which limits the generalization of the results across educational contexts. Consequently, the first goal of the future work will be to extend the development of the tool by collaborating with other schools at national and international levels. As ProgTutor was designed to train students of any level in the fundamentals of Python programming, it is important to broaden the range of application to higher education.

Other key aspect for the future will be to improve the learning path by including a machine learning model for exercise assignment with the aim of achieving a higher personalization level. To this end, a reinforcement learning model is already under development. A third line of future improvement, derived from the student's feedback, will be the improvement of the

explainability of the system. On the one hand, learners will understand why a new exercise was assigned and teachers will see how metrics led to specific suggestions—strengthening appropriate reliance on automated reckoning while further empowering human judgment. On the other hand, how the feedback information, accompanying hints, and error messages are presented to the student must also be improved.

To sum up, the transformation of the learning landscapes that will come up in the following years due to the integration of AI-based tools in education will be really relevant. ITSs and adaptive learning systems have the potential to augment students' and teachers' capacities, allowing them to achieve new levels of mastery. But, from the current experience with ProgTutor, it became clear that there is still a lot of research to do to really understand the implications of these new tools on learning, and how to integrate in classrooms while maintaining the teacher in the center of the pedagogical process.

REFERENCES

- [1] A. Shemshack and J. M. Spector, "A systematic literature review of personalized learning terms," *Smart Learn. Environ.*, vol. 7, no. 1, 2020, Art. no. 33.
- [2] T. Kabudi, I. Pappas, and D. H. Olsen, "AI-enabled adaptive learning systems: A systematic mapping of the literature," *Comput. Educ.: Artif. Intell.*, vol. 2, 2021, Art. no. 100017.
- [3] A. Maedche et al., "AI-based digital assistants: Opportunities, threats, and research perspectives," *Bus. Inf. Syst. Eng.*, vol. 61, pp. 535–544, 2019.
- [4] E. Mousavinasab, N. Zarifasanaiey, S. R. N. Kalhori, M. Rakhshan, L. Keikha, and M. G. Saedi, "Intelligent tutoring systems: A systematic review of characteristics, applications, and evaluation methods," *Interact. Learn. Environ.*, vol. 29, no. 1, pp. 142–163, 2021.
- [5] L. Zhang, J. D. Basham, and S. Yang, "Understanding the implementation of personalized learning: A research synthesis," *Educ. Res. Rev.*, vol. 31, 2020, Art. no. 100339.
- [6] C. Panigutti et al., "The role of explainable AI in the context of the AI act," in *Proc. ACM Conf. Fairness, Accountability, Transp.*, 2023, pp. 1139–1150.
- [7] E. Scott, A. Soria, and M. Campo, "Adaptive 3D virtual learning environments—A review of the literature," *IEEE Trans. Learn. Technol.*, vol. 10, no. 3, pp. 262–276, Jul.–Sep. 2017.
- [8] L. Guo, D. Wang, F. Gu, Y. Li, Y. Wang, and R. Zhou, "Evolution and trends in intelligent tutoring systems research: A multidisciplinary and scientometric view," *Asia Pacific Educ. Rev.*, vol. 22, no. 3, pp. 441–461, 2021.
- [9] D. Treceño-Fernández, J. C. del Campo, M. L. Bote-Lorenzo, E. Gómez-Sánchez, R. de Luis-García, and C. Alberola-López, "Integration of an intelligent tutoring system in a magnetic resonance simulator for education: Technical feasibility and user experience," *Comput. Methods Programs Biomed.*, vol. 195, 2020, Art. no. 105634.
- [10] R. Hare, Y. Tang, and C. Zhu, "Combining gamification and intelligent tutoring systems for engineering education," in *Proc. IEEE Front. Educ. Conf.*, 2023, pp. 1–5.
- [11] C. Dede, A. Etemadi, and T. Forshaw, "Intelligence augmentation: Upskilling humans to complement AI," Next Level Lab, Harvard Graduate School of Education, Cambridge, MA, USA, 2021. [Online]. Available: <https://projects.iq.harvard.edu/files/ai-ed2018/files/intelligence-augmentation-upskilling-humans-to-complement-ai.pdf>
- [12] S. Shen et al., "A survey of knowledge tracing: Models, variants, and applications," *IEEE Trans. Learn. Technol.*, vol. 17, pp. 1898–1919, 2024.
- [13] T. W. Price, Y. Dong, and D. Lipovac, "ISNAP: Towards intelligent tutoring in novice programming environments," in *Proc. ACM SIGCSE Tech. Symp. Comput. Sci. Educ.*, Seattle, WA, USA, 2017, pp. 483–488.
- [14] D. Roldán-Álvarez and F. J. Mesa, "Intelligent deep-learning tutoring system to assist instructors in programming courses," *IEEE Trans. Educ.*, vol. 67, no. 1, pp. 153–160, Feb. 2024.
- [15] P. Sharma and M. Harkishan, "Designing an intelligent tutoring system for computer programming in the Pacific," *Educ. Inf. Technol.*, vol. 27, no. 5, pp. 6197–6209, 2022.
- [16] P. Blikstein, M. Worsley, C. Piech, M. Sahami, S. Cooper, and D. Koller, "Programming pluralism: Using learning analytics to detect patterns in the learning of computer programming," *J. Learn. Sci.*, vol. 23, no. 4, pp. 561–599, 2014.
- [17] C. Charitsis, C. Piech, and J. C. Mitchell, "Feedback on program development process for CS1 students," in *Proc. 53rd ACM Tech. Symp. Comput. Sci. Educ.*, 2022, Art. no. 1150.
- [18] J. Yang and B. Zhang, "Artificial intelligence in intelligent tutoring robots: A systematic review and design guidelines," *Appl. Sci.*, vol. 9, no. 10, 2019, Art. no. 2078.
- [19] A. Alam, "Employing adaptive learning and intelligent tutoring robots for virtual classrooms and smart campuses: Reforming education in the age of artificial intelligence," in *Advanced Computing and Intelligent Technologies*. Berlin, Germany: Springer, 2022, pp. 395–406.
- [20] J. Collins, S. Chand, A. Vanderkop, and D. Howard, "A review of physics simulators for robotic applications," *IEEE Access*, vol. 9, pp. 51416–51431, 2021.
- [21] J. M. Cañas, E. Perdices, L. García-Pérez, and J. Fernández-Conde, "A ROS-based open tool for intelligent robotics education," *Appl. Sci.*, vol. 10, no. 21, 2020, Art. no. 7419.
- [22] C. Camargo, J. Gonçalves, M. A. Conde, F. J. Rodríguez-Sedano, P. Costa, and F. J. García-Peñalvo, "Systematic literature review of realistic simulators applied in educational robotics context," *Sensors*, vol. 21, no. 12, 2021, Art. no. 4031.
- [23] D. Scaradozzi, L. Cesaretti, L. Screpanti, and E. Mangina, "Identification and assessment of educational experiences utilizing data mining with robotics," *IEEE Robot. Autom. Mag.*, vol. 28, no. 4, pp. 103–113, Dec. 2021.
- [24] I. Jormanainen and E. Sutinen, "Using data mining to support teacher's intervention in a robotics class," in *Proc. 4th IEEE Int. Conf. Digit. Game Intell. Toy Enhanced Learn.*, 2012, pp. 39–46.
- [25] Coderz. Accessed: 9 Sep. 2024. [Online]. Available: <https://gocoderz.com/>
- [26] Codecombat. Accessed: 2 Oct. 2024. [online]. available: <https://codecombat.com/>
- [27] F. Bellas, M. Naya-Varela, A. Mallo, and A. Paz-Lopez, "Education in the AI era: A long-term classroom technology based on intelligent robotics," *Humanities Social Sci. Commun.*, vol. 11, 2024, Art. no. 1425.
- [28] RoboboSim simulator. Accessed: 9 Sep. 2024. [Online]. Available: <https://github.com/mintforpeople/robobo-programming/wiki/Unity>
- [29] W. Holmes, "Artificial Intelligence in education," in *Encyclopedia of Education and Information Technologies*. Berlin, Germany: Springer, 2019.
- [30] I. Hernán-Losada, J. Velázquez-Iturbide, and C. Lázaro-Carrascosa, "Programming learning tools based on Bloom's taxonomy: Proposal and accomplishments," in *Proc. 8th Int. Symp. Comput. Educ.*, 2006, pp. 24–26.
- [31] T.-T. Wu, A. Asmara, Y.-M. Huang, and I. P. Hapsari, "Identification of problem-solving techniques in computational thinking studies: Systematic literature review," *SAGE Open*, vol. 14, no. 2, 2024, Art. no. 21582440241249897.